

EXPT NO: 4	TRANSFER LEARNING USING PRE-TRAINED VISION MODELS FOR IMAGE RECOGNITION
DATE: 17/8/26	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. **What is transfer learning in deep learning?**

Transfer learning is a deep learning technique in which a model that has already been trained on a large dataset is reused for a new but related task. The pre-trained model already contains useful knowledge and features, so it can be adapted to the new problem instead of training a model completely from scratch. This saves time, computational resources, and data.

2. **What are the advantages of using pre-trained models over training from scratch?**

Pre-trained models have several advantages. They reduce training time and computational cost because the model has already learned important features. They require less training data and usually provide better accuracy, especially when the available dataset is small. They also make model development easier and allow the model to be quickly adapted to new tasks using fine-tuning.

3. **What is the role of fine-tuning in transfer learning?**

Fine-tuning is the process of further training a pre-trained model using a new dataset related to the target task. During fine-tuning, some or all of the model's layers are adjusted to learn features specific to the new problem. It helps improve the performance of the pre-trained model while retaining the useful knowledge it learned from the original dataset.

4. **Name any three popular pre-trained computer vision models.**

Three popular pre-trained computer vision models are:

- VGG16
- ResNet50
- InceptionV3

These models are commonly used for tasks such as image classification, object recognition, and feature extraction.

5. **What is the purpose of Hugging Face and how is it useful in deep learning?**

Hugging Face is an AI platform that provides a large collection of pre-trained models, datasets, and libraries for deep learning and artificial intelligence. Its Transformers library makes it easy to download, use, and fine-tune pre-trained models for tasks such as text classification, translation, question answering, image classification, and multimodal applications. It helps developers and researchers save time by using existing state-of-the-art models instead of building and training models from scratch.

AIM

To implement transfer learning using pre-trained vision models for image recognition and evaluate their performance on real-world image datasets.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow/Keras or PyTorch
- Google Colab
- Hugging Face Transformers
- Kaggle Dataset

- Matplotlib
- GitHub

OBJECTIVES

Students will be able to:

- Understand the concept of transfer learning.
- Apply pre-trained vision models for image recognition.
- Fine-tune models such as ResNet or AlexNet.
- Utilize Kaggle datasets and Hugging Face vision models.
- Compare transfer learning performance with custom CNN models.

PROCEDURE

PART A: DATASET PREPARATION

Perform the following tasks:

- Download an image dataset from Kaggle.
- Load and preprocess the images.
- Split the dataset into training, validation, and testing sets.

PART B: IMPLEMENTING TRANSFER LEARNING

Load a pre-trained model such as:

- ResNet-50
- AlexNet

Replace the final classification layer according to the dataset classes.

Freeze the feature extraction layers.

Compile the model.

PART C: MODEL TRAINING AND PERFORMANCE EVALUATION

Train the transfer learning model.

Record:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss
- Test accuracy

Visualize:

- Accuracy vs Epoch
- Loss vs Epoch

PART D: USING HUGGING FACE PRE-TRAINED MODELS

Perform the following tasks:

- Load a pre-trained vision model from Hugging Face.
- Perform image classification on sample images.
- Compare predictions with the transfer learning model.
- Document observations and upload the notebook to GitHub.

24BAD088 DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

24BAD088 Pradeep Manikandan D

Intel Image Classification

This notebook is optimized for faster Google Colab execution.

Dataset: <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

Model: MobileNetV2 (lightweight pre-trained vision model) + Hugging Face ViT

Speed settings: 160x160 images, batch size 64, 2 epochs, frozen base model, prefetch.

[1] `!pip install -q kagglehub transformers` Python

24BAD088 DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

`import os, random`
`import numpy as np`
`import matplotlib.pyplot as plt`
`import tensorflow as tf`

`from tensorflow.keras.applications import MobileNetV2`
`from tensorflow.keras.applications.mobilenet_v2 import preprocess_i`
`from tensorflow.keras.layers import GlobalAveragePooling2D, Dense,`
`from tensorflow.keras.models import Model`
`from tensorflow.keras.optimizers import Adam`

`print("TensorFlow:", tf.__version__)`
`print("GPU:", tf.config.list_physical_devices("GPU"))`

[2] Python
TensorFlow: 2.20.0
GPU: []

`import kagglehub`

`path = kagglehub.dataset_download("puneet6060/intel-image-classific`
`train_dir = os.path.join(path, "seg_train", "seg_train")`
`test_dir = os.path.join(path, "seg_test", "seg_test")`

[3] Python

Cell 1 of 17

24BAD088 DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

`import kagglehub`

`path = kagglehub.dataset_download("puneet6060/intel-image-classific`
`train_dir = os.path.join(path, "seg_train", "seg_train")`
`test_dir = os.path.join(path, "seg_test", "seg_test")`

`print("Dataset:", path)`
`print("Classes:", sorted(os.listdir(train_dir)))`

[3] Python
Using Colab cache for faster access to the 'intel-image-classification'
Dataset: /kaggle/input/intel-image-classification
Classes: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']

`IMG_SIZE = (160, 160)`
`BATCH_SIZE = 64`
`EPOCHS = 2`
`SEED = 42`

`train_ds = tf.keras.utils.image_dataset_from_directory(`
`train_dir, validation_split=0.2, subset="training",`
`seed=SEED, image_size=IMG_SIZE, batch_size=BATCH_SIZE,`
`label_mode="categorical"`
`)`

[4] Python
Found 14034 files belonging to 6 classes.
Using 11228 files for training.
Found 14034 files belonging to 6 classes.
Using 2806 files for validation.
Found 3000 files belonging to 6 classes.
Classes: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']

24BAD088 DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

`val_ds = tf.keras.utils.image_dataset_from_directory(`
`train_dir, validation_split=0.2, subset="validation",`
`seed=SEED, image_size=IMG_SIZE, batch_size=BATCH_SIZE,`
`label_mode="categorical"`
`)`

`test_ds = tf.keras.utils.image_dataset_from_directory(`
`test_dir, image_size=IMG_SIZE, batch_size=BATCH_SIZE,`
`label_mode="categorical", shuffle=False`
`)`

`class_names = train_ds.class_names`
`AUTOTUNE = tf.data.AUTOTUNE`
`train_ds = train_ds.prefetch(AUTOTUNE)`
`val_ds = val_ds.prefetch(AUTOTUNE)`
`test_ds = test_ds.prefetch(AUTOTUNE)`

`print("Classes:", class_names)`

[4] Python
Found 14034 files belonging to 6 classes.
Using 11228 files for training.
Found 14034 files belonging to 6 classes.
Using 2806 files for validation.
Found 3000 files belonging to 6 classes.
Classes: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']

Cell 1 of 17

24BAD088_DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

```
# Sample images
plt.figure(figsize=(10, 7))
for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[np.argmax(labels[i].numpy())])
        plt.axis("off")
plt.tight_layout()
plt.show()
```

Python

mountain

street

forest

buildings

sea

forest

24BAD088_DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

buildings

sea

forest

Part B – Transfer Learning

MobileNetV2 is used instead of ResNet50 because it is much lighter and faster in Google Colab while still demonstrating transfer learning with a pre-trained vision model.

```
base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(160, 160, 3)
)
base_model.trainable = False
```

Python

24BAD088_DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

```
base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(160, 160, 3)
)
base_model.trainable = False

inputs = tf.keras.Input(shape=(160, 160, 3))
x = preprocess_input(inputs)
x = base_model(x, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation="relu")(x)
x = Dropout(0.3)(x)
outputs = Dense(len(class_names), activation="softmax")(x)

model = Model(inputs, outputs)

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

print("Frozen layers:", len(base_model.layers))
model.summary()
```

Python

24BAD088_DL_Expt-4.ipynb

Generate + Code + Markdown Run All ...

Select Kernel

Downloading data from <https://storage.googleapis.com/tensorflow/keras-9406464/9406464>

0s 0us/step

Frozen layers: 154

Model: "functional"

Layer (type)	Output Shape	Pa
input_layer_1 (InputLayer)	(None, 160, 160, 3)	
true_divide (TrueDivide)	(None, 160, 160, 3)	
subtract (Subtract)	(None, 160, 160, 3)	
mobilenetv2_1.00_160 (Functional)	(None, 5, 5, 1280)	2,25
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	
dense (Dense)	(None, 128)	16
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 6)	

```

24BAD088_DL_Expt-4.ipynb
Generate + Code + Markdown Run All ... Select Kernel

Total params: 2,422,726 (9.24 MB)

Trainable params: 164,742 (643.52 KB)

Non-trainable params: 2,257,984 (8.61 MB)

# Fast training
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

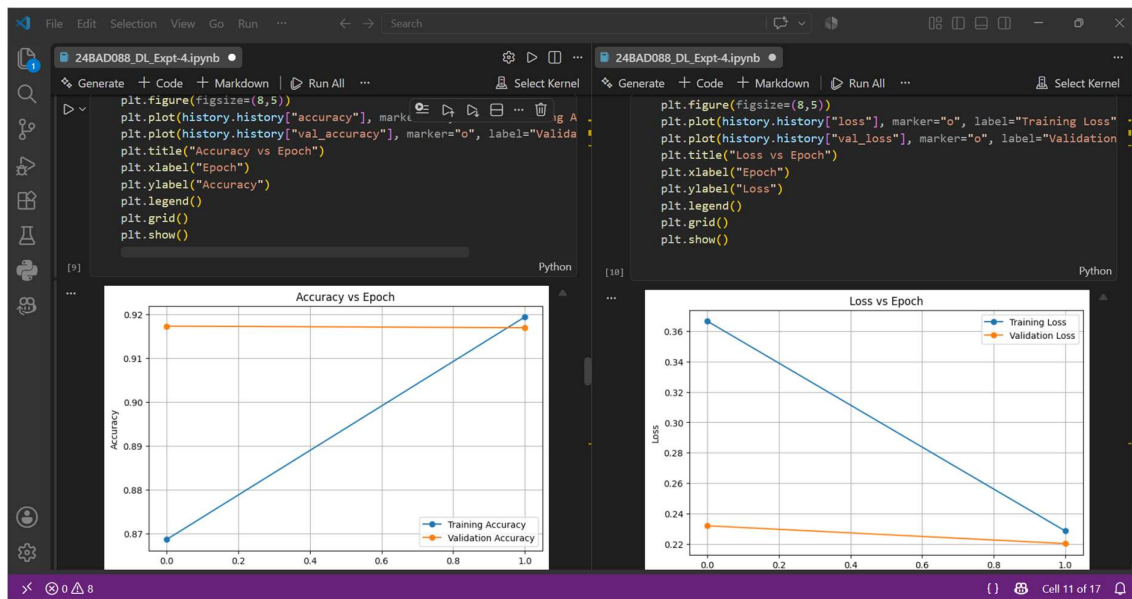
Epoch 1/2
176/176 360s 2s/step - accuracy: 0.8686 - loss: 0.
Epoch 2/2
176/176 342s 2s/step - accuracy: 0.9194 - loss: 0.

# Test performance
test_loss, test_accuracy = model.evaluate(test_ds)
print("\nRESULTS")
print("-" * 40)
print("Training Accuracy : ", round(history.history["accuracy"][-1], 4))
print("Validation Accuracy : ", round(history.history["val_accuracy"][-1], 4))
print("Training Loss : ", round(history.history["loss"][-1], 4))
print("Validation Loss : ", round(history.history["val_loss"][-1], 4))
print("Test Accuracy : ", round(test_accuracy, 4))

RESULTS
-----
Training Accuracy : 0.9194
Validation Accuracy : 0.917
Training Loss : 0.2286
Validation Loss : 0.2204
Test Accuracy : 0.9163

# Accuracy graph
plt.figure(figsize=(8,5))
plt.plot(history.history["accuracy"], marker="o", label="Training Accuracy")
plt.plot(history.history["val_accuracy"], marker="o", label="Validation Accuracy")
plt.legend()
plt.grid()
plt.show()

```



```

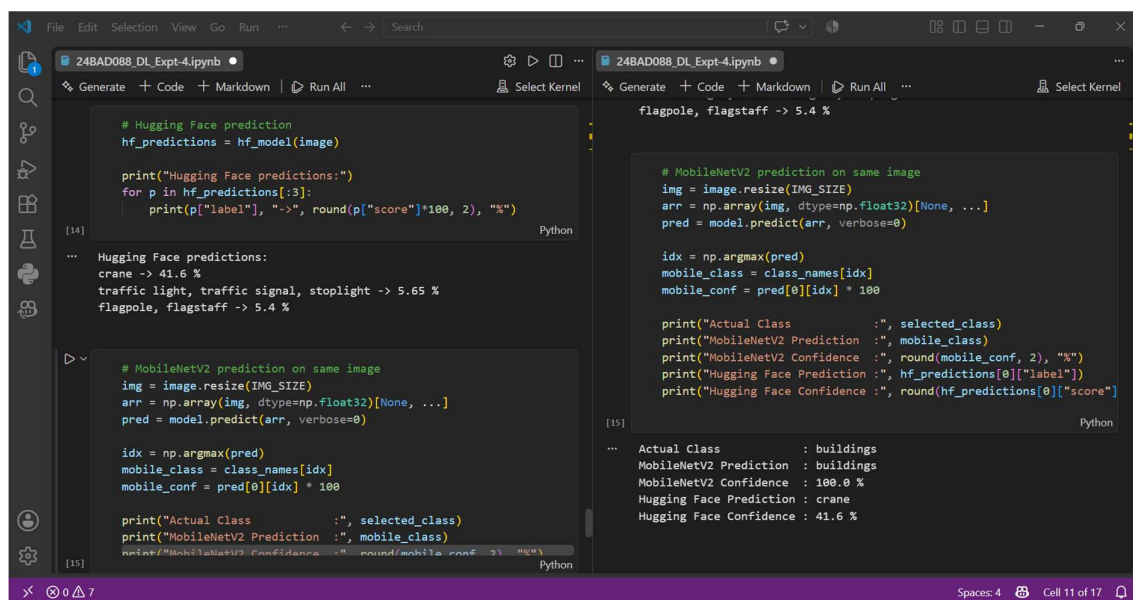
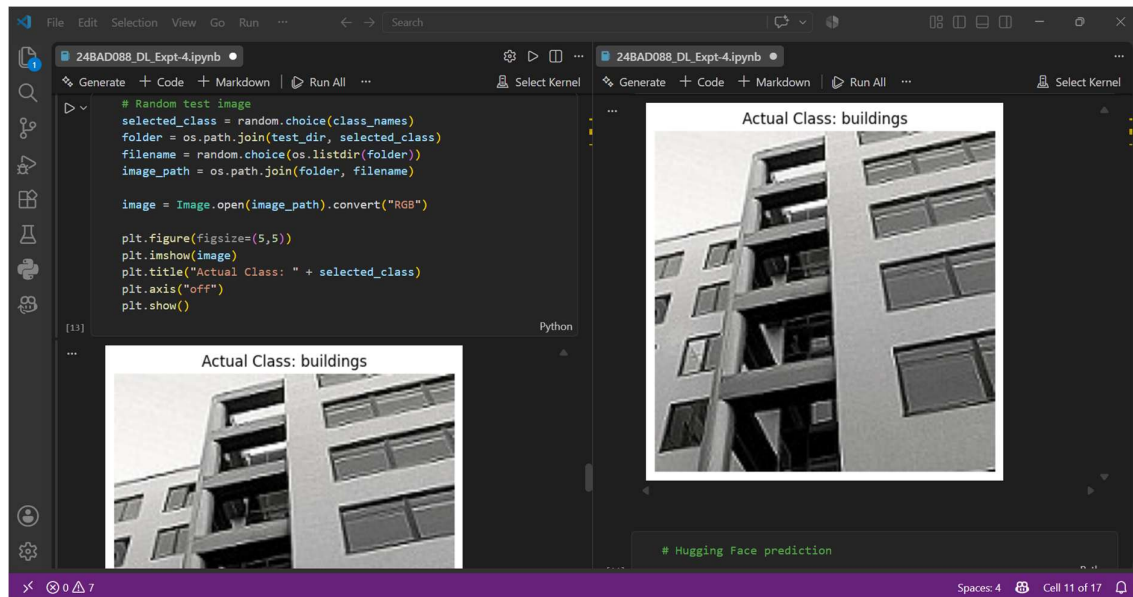
[12] 5s
from transformers import pipeline

hf_model = pipeline(
    "image-classification",
    model="google/vit-base-patch16-224"
)

print("Hugging Face Vision Transformer loaded successfully!")

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 69.7k/69.7k [00:00<00:00, 1.88MB/s]
model.safetensors: reconstructing file: 100% 346MB / 346MB, 30.3MB/s
model.safetensors: downloading bytes: 328MB, 29.7MB/s
Loading weights: 100% 200/200 [00:00<00:00, 3651.67kB/s]
preprocessor_config.json: 100% 160/160 [00:00<00:00, 13.9kB/s]
Hugging Face Vision Transformer loaded successfully!

```



EXPECTED OUTCOME

Students will understand the advantages of transfer learning and gain practical experience in fine-tuning pre-trained vision models from TensorFlow/PyTorch and Hugging Face for image recognition tasks.

GitHub: https://github.com/Pradeep-Manikandan-1-D/24BAD088_DEEP_LEARNING_LAB

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. **What are the advantages of transfer learning compared to training a CNN from scratch?**

Transfer learning reduces training time and computational requirements because the model is already trained on a large dataset. It requires less training data and can achieve higher

accuracy with fewer epochs. It also reduces the risk of overfitting and allows the model to use useful features learned from the original dataset.

2. **Why are pre-trained models able to achieve higher accuracy with fewer training epochs?**

Pre-trained models have already learned important features such as edges, textures, shapes, and patterns from large and diverse datasets. Therefore, when applied to a new task, the model does not need to learn these basic features again. Only task-specific features need to be learned, allowing the model to achieve good accuracy in fewer training epochs.

3. **Compare the performance of the transfer learning model with the CNN implemented in the previous experiment.**

The transfer learning model generally provides better performance than a CNN trained from scratch. It usually achieves higher validation and test accuracy, lower training time, and faster convergence. The CNN trained from scratch requires more epochs and a larger amount of data to learn useful features. Therefore, transfer learning is more efficient and suitable when the available dataset is limited.

4. **What is the purpose of freezing layers during transfer learning?**

Freezing layers prevents their weights from being updated during training. The early layers of a pre-trained model usually contain general features such as edges, textures, and shapes that are useful for many image-related tasks. By freezing these layers, the already learned knowledge is preserved, training becomes faster, and the risk of overfitting is reduced. The later layers can then be fine-tuned for the new task.

5. **How can Hugging Face simplify the deployment and experimentation of vision models?**

Hugging Face provides a large collection of pre-trained vision models, datasets, and ready-to-use libraries. Users can easily download and test models without implementing them from scratch. Models can also be fine-tuned on custom datasets and used for tasks such as image classification, object detection, and image generation. Its tools and model repository make experimentation, comparison, sharing, and deployment of vision models easier and faster.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		